QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

TEAM NAME: Its Works On My Machine

TEAM CODE: 106

TEAM MEMBERS	EMAIL
EDISON KONG BANG JEAN (LEADER)	edisonkong05@gmail.com
DARREN WONG CHEE LIANG	darren05wong@gmail.com
GOH JUN TECK	teck0423@gmail.com
LIU YOU ZHENG	liuyz8715@gmail.com
LIEW TIAN EN	ahenahyi05052004@gmail.com

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	SynapseCore System (AI-Powered Agentic Workflow Automation for HQ and Branch Coordination)
Team Repo URL	https://github.com/EdisonShiny/synapsecore-system.git
Project Board URL	https://docs.google.com/document/d/1cWZ2zkrP0Fvw6nrY6P0U549u3wN_6v5pus2IfNeGjL0/edit?usp=sharing
Live Deployment URL (optional for preliminary round)	N/A

Objective:

The main objective is to ensure that the SynapseCore System can support coordination between Headquarters (HQ) and Branch Offices reliably through AI-powered workflows.

The system should:

- Process unstructured inputs into structured workflows
- Generate projects using AI
- Allow HQ to approve or reject decisions
- Allow Branch to submit requests and update progress
- Maintain workflow phases correctly
- Reduce AI errors using validation steps

The system also aims to:

- Reduce communication gaps between HQ and Branch
- Enable smooth task handover
- Automate multi-step business processes
- Improve decision speed and quality
- Adapt to real-world business changes

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section defines what features are tested and what are not included.

1.1 In-Scope Core Features

- User Authentication: Login for HQ and Branch
- Workflow System: Create and run workflows
- AI Processing: Generate report, extract data, validate output
- Project Management: Create and manage projects
- Approval Flow: HQ approve or reject
- Request System: Branch submit and reapply requests
- Phase Progression: Move project through phases
- Database: View and edit structured data
- Settings: Update user and AI configuration

1.2 Out-of-Scope

- App themes
- UI customization
- Full production deployment testing

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI produces wrong or fake output	4	5	20 (Critical)	Use validation and retry logic	Test with incorrect input
Branch accesses HQ-only actions	3	5	15 (High)	Role-based access control	Try performing HQ actions as Branch
Workflow creates wrong project	3	5	15 (High)	Add validation before project creation	Run workflow with test input
Request fails validation	3	5	15 (High)	Block invalid requests safely	Submit weak request

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
System build failure	2	5	10 (Medium)	Require build pass before release	Run build command
Database update breaks data	2	5	10 (Medium)	Restrict editable fields	Modify test data

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
    - Scope:
        - AI output format
        - Data validation
    - Execution: Tested manually using small inputs
    - Pass Condition: Output is correct and structured

- **Integration Test**
    - Scope: Full workflow process
    - Execution: Input → AI → Project → Approval
    - Pass Condition: All steps complete successfully

- **Test Environment (CI/CD Practice):**
    - Local Testing: http://localhost:3000
    - No CI/CD pipeline available
    - Testing done manually

- **Regression Testing & Pass/Fail Rules:**
    - Test critical features before demo
    - Pass = expected result matches actual
    - Fail = wrong result or system error

- **Test Data Strategy:**
    - Demo HQ and Branch accounts
    - Sample workflows
    - Simple text inputs
    - Small file uploads
    - Sample prompt for workflow

- **Passing Rate Threshold:**
    - Critical tests: 100% pass required
    - Overall system: minimum 85% pass

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	**Requirements**	**Project**	**Pass/Failed**
Build	**Zero Error**	**Passed**	**Passed**
Unit Tests	**100% Passing Rate**	**Manual only**	**Passed**
Code Quality	**Zero Lint Error**	**Passed with warnings**	**Passed**
Test Coverage	**Minimum 80%**	**N/A**	**N/A**

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	**≥ 90%**	**100%**	**Passed**
AI Output Pass Rate	**≥ 80%**	**100%**	**Passed**
Critical Bugs	**0**	**0**	**Passed**
API Performance	**< 800ms**	**~500ms avg**	**Passed**
Security	**API key not exposed**	**Clean**	**Passed**

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow: Workflow → Project → Approval)	Verify full system flow works correctly	Login as HQ Create workflow Input: "Stock shortage in Penang branch" Run workflow Check project list Approve project Submit phase outcome	Workflow completes, project created, approved, and progressed	Workflow executed successfully 2 projects generated HQ approved project Phase progressed correctly Status: Passed
TC-02	Specific Case	System handles	Input: "Something	System rejects or	AI validation

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
	(Negative: Invalid Input)	unclear input safely	wrong” Run workflow	asks for clarification	failed System retried 3 times Workflow stopped safely No incorrect project created Status: PASSED
TC-03	NFR (Performance Test)	System performance under usage	Run multiple workflows Observe response time	Response time < 800ms	Average response ~500ms No crash observed Status: PASSED

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	**Prompt Input**	**Expected Output (Acceptance Criteria)**	**Actual Output**	**Status**
AI-01	Clear business problem	Structured workflow	Correct structured output generated	Passed
AI-02	Incomplete input	Ask for more info	Validation failed and stopped	Passed
AI-03	Noisy input	No hallucination	Relevant output only, no fake data	Passed

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	Not defined
Input used while testing	Large text (~5000 words)
Expected Behavior	System handles or rejects
Actual Behavior	• System processed input in mock mode • No crash observed
Status	PASSED

6.3. Adversarial/Edge Prompt Test

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

- Input: Ambiguous instruction
- Expected Behavior: Ask for clarification
- Actual Behavior:
  - AI failed validation
  - System stopped workflow safely
- Status: PASSED

6.4. Hallucinating Handling

This section is defined for you to describe how your system does detect or mitigate hallucinating response.

- System approach:
  - Extract key information
  - Validate against input
  - Retry up to 3 times
  - Reject invalid output
- Result:
  - No hallucinated output accepted
  - All incorrect outputs blocked
- Status: PASSED

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.